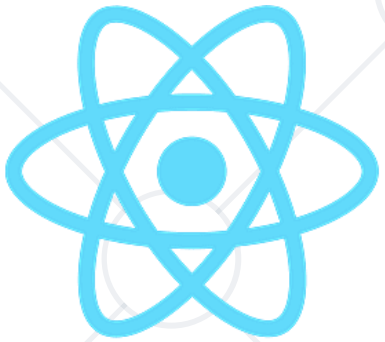


State Management and Data Persistence

Beyond Hardcoded Data: Building Scalable Apps with
State Management and APIs



React Native

SoftUni Team
Technical Trainers



SoftUni



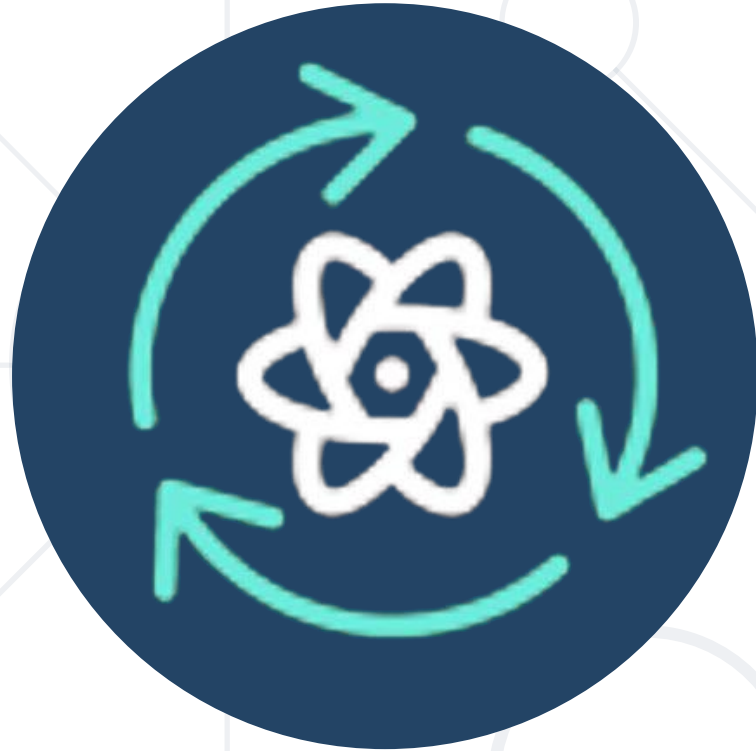
Software University

<https://softuni.bg>

Table of Contents

1. State Fundamentals
2. Global State & Context API
3. Data Persistence
4. Fetching Remote Data
5. Optimized List Rendering





State Fundamentals

What is State?

- 
- State represents **data that changes over time** in your application
 - Determines what is **rendered** on the screen
 - **Triggers** re-renders when updated
 - Lives in memory (by default)
 - Can be **local**, **shared**, or **persisted**
 - Drives **UI behavior** and logic

State is the single source of truth for your UI.

- Different kinds of state exist at different levels of an app
 - **Navigation State**
 - **Local State**
 - **Shared State**
 - **Global State**
 - **Remote State**

- State owned by a **single component**
 - Managed with **useState**
 - Scoped to one component
 - Easy to reason about
 - Resets when component unmounts
 - Ideal for UI-specific data

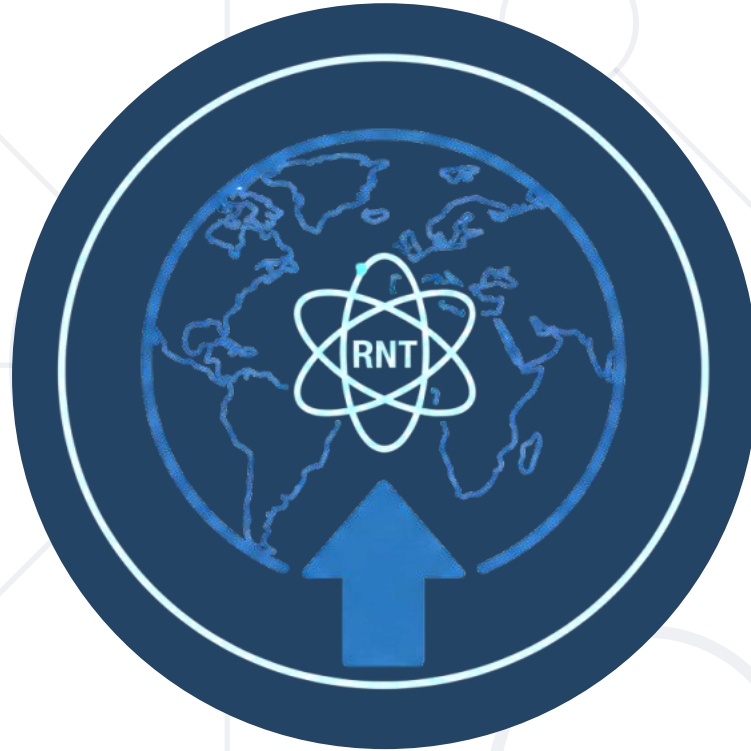
```
const [count, setCount] = useState(0);
```

Local state is simple, fast, and isolated.

When Local State Is Not Enough

- Problems start when state needs to be **shared**
 - **Multiple** components need the same data
 - **Props drilling** becomes messy
 - State duplication causes **bugs**
 - Synchronization becomes **hard**
 - App logic becomes **fragile**

Shared state requires a different solution.



Global State & Context API

What Is Global State?

- 
- **Global state** is shared across multiple parts of the app
 - **Accessible** by many screens
 - **Independent** of component hierarchy
 - **Centralized** source of truth
 - Useful for app-wide data
 - Lives as long as the app **runs**

Global state solves cross-screen data sharing.

- Context provides a way to pass data **without props drilling**
 - Built into React
 - Provider → Consumer model
 - Works with **hooks**
 - **No external** libraries required
 - Best for **low-frequency** updates

```
const ThemeContext = createContext();
```

Context simplifies data sharing across the tree.

- The Provider makes state **available** to its children
 - **Wraps** part (or all) of the app
 - **Holds** shared state
 - **Controls** who can access it
 - Can expose **state + actions**
 - **Triggers** re-renders on change

```
<ThemeContext.Provider value={{ theme }}>  
  {children}  
</ThemeContext.Provider>
```

- Components read **context** using hooks
 - **useContext** hook
 - **No prop** passing
 - **Cleaner** component APIs
 - Reacts to context **updates**
 - Easy to overuse

```
const { theme } = useContext(ThemeContext);
```

Context consumption is simple—but powerful.

- **useReducer** is a React Hook for managing **more complex local state logic**.
 - Alternative to **useState** for complex updates
 - State changes are driven by **actions**
 - Updates are centralized in a **reducer function**
 - Improves **readability** for multi-step logic
 - Commonly used with forms or **complex UI flows**

```
const [state, dispatch] = useReducer(reducer, initialState);
```

useReducer brings Redux-like structure to local component state.



Data Persistence

What Is AsyncStorage?

- AsyncStorage is a **key–value storage system** for React Native
 - **Persistent** across app restarts
 - Asynchronous API
 - String-based storage
 - Similar to **localStorage**
 - Device-level storage

AsyncStorage keeps data between sessions.

- Data is stored as **strings** under a **key**
 - Uses **setItem**
 - **Async** by nature
 - Manual serialization required
 - Can store small datasets
 - Error handling **required**



```
await AsyncStorage.setItem( 'user', JSON.stringify(user) );
```

Always serialize before storing.

- Reading data happens **asynchronously**
 - Uses **getItem**
 - Returns **null** if missing
 - Needs **parsing**
 - Usually done on **app start**
 - Often combined with **useEffect**

```
const data = await AsyncStorage.getItem('user');
```

Persistence requires async thinking.

Removing & Clearing Data

- Storage must be actively **managed**
 - **removeItem** deletes a single key
 - Clear wipes all storage
 - Common for logout flows
 - Must be handled carefully
 - **Irreversible** action



Clearing storage is part of app lifecycle.

What Should Be Stored?

- Not all state belongs in AsyncStorage
 - User preferences ✓
 - Onboarding flags ✓
 - UI state ✗
 - Large datasets ✗
 - Sensitive data (unencrypted) ✗
 - Auth tokens ?

Persist intent, not UI noise.

- **AsyncStorage** should be used intentionally and sparingly
 - Store **small, essential data only**
 - Always **serialize / deserialize**
 - Use **clear, consistent key naming**
 - Handle **errors and null values**
 - Load data **once on app start**, not on every render
 - **Avoid** storing sensitive data unencrypted

AsyncStorage is for persistence, not for state management.



Fetching Remote Data

Why Fetch Data?

- 
- Most apps depend on **remote data**
 - **APIs** power modern apps
 - Data comes from **servers**
 - Network is unreliable
 - Requests are **asynchronous**
 - UI must handle **all states**

Data fetching is a core app concern.

- **Fetch** is built into React Native
 - Promise-based
 - Minimal API
 - **No automatic** JSON parsing
 - **Manual** error handling
 - Standard web API

```
const res = await fetch(url);  
const data = await res.json();
```

Fetch is simple but low-level.

- **Axios** is a popular HTTP client
 - **Automatic** JSON parsing
 - Interceptors support
 - **Better** error handling
 - **Cleaner** syntax
 - **External** dependency

A X ↕ O S

```
const { data } = await axios.get(url);
```

Axios improves developer experience.

- Fetching is **never** just "data"
 - Loading state **required**
 - Error state **required**
 - Empty state **required**
 - UI must react to **all cases**
 - Users **hate** frozen screens

```
if (loading) return <Loader />;
```

Fetching is a state machine, not a call.

- Placement **matters**
 - **useEffect** for lifecycle fetching
 - Triggered on screen **mount**
 - Triggered by user **actions**
 - **Avoid** fetching in render
 - Keep side effects **isolated**

Data fetching belongs to effects.



Optimized List Rendering

Why FlatList?



- Rendering lists efficiently is **hard**
 - Mobile performance **constraints**
 - **Large** datasets
 - Memory usage **matters**
 - Scroll performance is **critical**
 - **Naive** rendering breaks apps

Lists require specialized components.

- **FlatList** is optimized for **large** lists
 - Virtualized rendering
 - Renders only **visible** items
 - Scroll-friendly
 - Highly configurable
 - Production-ready

```
<FlatList data={items} renderItem={...} />
```

FlatList is the default list solution.

- **FlatList** requires specific **props**
 - **data** – array of items
 - **renderItem** – how to render each item
 - **keyExtractor** – unique keys
 - **ListEmptyComponent**
 - **ListFooterComponent**



Keys and renderItem define list identity.

- Each item is rendered via a **function**
 - Receives **{ item, index }**
 - Must return **JSX**
 - Should be lightweight
 - Avoid inline heavy logic
 - Can be extracted to components

```
renderItem=(({ item }) => <Itemdata={item} />)
```

Keep renderItem fast and simple.

- They are not **interchangeable**
 - ScrollView renders **everything**
 - FlatList renders **lazily**
 - ScrollView → **static** / small lists
 - FlatList → **dynamic** / large lists
 - Performance difference is **massive**

Use FlatList by default for lists.

- **State as Truth**: State drives the UI—from simple local toggles to complex global logic
- Use **Context API** to eliminate prop drilling and centralize shared data.
- Use **AsyncStorage** to keep critical data (like tokens) alive across app restarts
- Always prioritize **FlatList** over **ScrollView** for dynamic lists to ensure mobile performance.



- Software University – High-Quality Education, Profession and Job for Software Developers

- softuni.bg

- Software University Foundation

- softuni.foundation

- Software University @ Facebook

- facebook.com/SoftwareUniversity



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg/>
- © Software University – <https://softuni.bg>

